```
"""
DecodeLabs Industrial Training Kit - Project 1
Password Strength Checker
Track: Defensive Logic | Junior Analyst

Goal:
    Create a program that checks whether a password is weak, medium,
    or strong, based on length, character variety, and known-leaked
    password lists.

Key Skills demonstrated:
    - String handling
    - Conditional logic
    - Security basics (entropy, common-password blocklisting)
"""

import getpass

# A small sample of commonly leaked / breached passwords.
# In a production system this list would be sourced from a
# proper breach database (e.g. Have I Been Pwned's Pwned Passwords API).
COMMON_LEAKED_PASSWORDS = {
    "123456", "123456789", "qwerty", "password", "111111",
    "12345678", "abc123", "1234567", "password1", "12345",
    "iloveyou", "admin", "welcome", "monkey", "letmein",
    "football", "qwerty123", "dragon", "password123", "sunshine",
}

MIN_LENGTH = 8


def check_leaked(password: str) -> bool:
    """Return True if the password appears in the known-leaked list."""
    return password.lower() in COMMON_LEAKED_PASSWORDS


def check_length(password: str) -> bool:
    """Return True if the password meets the minimum length requirement."""
    return len(password) >= MIN_LENGTH


def check_character_variety(password: str) -> dict:
    """
    Scan the password once (O(n)) and report which character
```

```
    classes are present using Python's built-in, C-optimized
    string methods rather than manual loops.
    """
    return {
        "has_upper": any(char.isupper() for char in password),
        "has_lower": any(char.islower() for char in password),
        "has_digit": any(char.isdigit() for char in password),
        "has_symbol": any(not char.isalnum() for char in password),
    }


def classify_strength(password: str) -> tuple[str, list[str]]:
    """
    Classify a password as Weak, Medium, or Strong.

    Returns a tuple of (strength_label, list_of_feedback_notes).
    """
    notes = []

    # --- Gatekeeper checks: automatic fail conditions -----------------
    if check_leaked(password):
        notes.append("This password appears in a known breach/leak list.")
        return "Weak", notes

    if not check_length(password):
        notes.append(f"Too short: needs at least {MIN_LENGTH} characters.")
        return "Weak", notes

    # --- Character variety score --------------------------------------
    variety = check_character_variety(password)
    score = sum(variety.values())  # up to 4: upper, lower, digit, symbol

    if not variety["has_upper"]:
        notes.append("Add at least one uppercase letter.")
    if not variety["has_lower"]:
        notes.append("Add at least one lowercase letter.")
    if not variety["has_digit"]:
        notes.append("Add at least one number.")
    if not variety["has_symbol"]:
        notes.append("Add at least one symbol (e.g. !@#$%).")

    # Bonus: reward extra length beyond the minimum
    length_bonus = len(password) >= 12

    if score == 4 and length_bonus:
        return "Strong", ["Great password!"]
    elif score >= 3:
```

```
        return "Medium", notes
    else:
        return "Weak", notes


def display_result(password: str) -> None:
    """Run the checker on a password and print a formatted result."""
    strength, notes = classify_strength(password)

    bar = {"Weak": "[#---------]", "Medium": "[#####-----]", "Strong": "[########
    print(f"\nPassword Strength: {strength} {bar.get(strength, '')}")
    if notes:
        print("Feedback:")
        for note in notes:
            print(f"  - {note}")


def main():
    print("=" * 50)
    print(" DecodeLabs - Password Strength Checker")
    print("=" * 50)
    try:
        password = getpass.getpass("Enter a password to check: ")
    except Exception:
        # Fallback for environments where getpass isn't supported
        password = input("Enter a password to check: ")

    display_result(password)


if __name__ == "__main__":
    main()
```